

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
High Performance Computing (CSDS125)
Lab Assignment 5
Parallel Genetic Algorithm and Particle Swarm Optimization for Distributed Task Allocation using OpenMP

1 Problem Statement

In distributed computing systems, the **Task Allocation Problem (TAP)** involves assigning M cooperating tasks onto P processing nodes ($P \ll M$) in a manner that minimizes total system cost. The cost model comprises two components:

1. **Execution Cost (X)**: The cost incurred by executing task i on processor node p .
2. **Inter-task Communication Cost (C)**: The cost incurred by communication between tasks i and j . This penalty is incurred **only when interacting tasks are placed on distinct processors**.

Formally, let assignment vector $A = [A_0, A_1, \dots, A_{M-1}]$ where $A_i \in \{0, 1, \dots, P-1\}$ represents the processor index assigned to task i . The total cost function is:

$$\text{Total Cost} = \sum_{i=0}^{M-1} X_{i,A_i} + \sum_{i=0}^{M-1} \sum_{j=i+1}^{M-1} C_{i,j} \cdot [A_i \neq A_j] \quad (1)$$

where $[A_i \neq A_j]$ is an indicator variable returning 1 if task i and task j are mapped to different nodes, and 0 if co-located on the same node.

Because the solution space comprises P^M possible combinations, the problem is NP-hard. We implement and evaluate two parallel metaheuristic search algorithms accelerated with OpenMP: **Parallel Genetic Algorithm (P-GA)** and **Parallel Particle Swarm Optimization (P-PSO)**.

2 Verification of Sample Topology

To validate the cost evaluation logic, we model the standard sample topology ($M = 6$ tasks, $P = 2$ nodes) provided in the assignment specification:

2.1 Matrix Definitions

- **Inter-task Communication Matrix C (6×6)**:

$$C = \begin{bmatrix} 0 & 6 & 4 & 0 & 0 & 12 \\ 6 & 0 & 8 & 12 & 3 & 0 \\ 4 & 8 & 0 & 0 & 11 & 0 \\ 0 & 12 & 0 & 0 & 5 & 0 \\ 0 & 3 & 11 & 5 & 0 & 0 \\ 12 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- **Execution Cost Matrix X (6×2):**

$$X = \begin{bmatrix} t_1 : & 5 & 10 \\ t_2 : & 2 & \infty \\ t_3 : & 4 & 4 \\ t_4 : & 6 & 3 \\ t_5 : & 5 & 2 \\ t_6 : & \infty & 4 \end{bmatrix}$$

2.2 Cost Calculations

1. **Serial Assignment** ($t_1, t_2, t_3 \rightarrow n_1; t_4, t_5, t_6 \rightarrow n_2$):

$$\text{Execution Cost } (x) = x_{11} + x_{21} + x_{31} + x_{42} + x_{52} + x_{62} = 5 + 2 + 4 + 3 + 2 + 4 = 20$$

$$\begin{aligned} \text{Communication Cost } (c) &= c_{14} + c_{15} + c_{16} + c_{24} + c_{25} + c_{26} + c_{34} + c_{35} + c_{36} \\ &= 0 + 0 + 12 + 12 + 3 + 0 + 0 + 11 + 0 = 38 \end{aligned}$$

$$\text{Total Cost} = 20 + 38 = 58$$

2. **Optimal Assignment** ($t_1, t_2, t_3, t_4, t_5 \rightarrow n_1; t_6 \rightarrow n_2$):

$$\text{Execution Cost } (x) = x_{11} + x_{21} + x_{31} + x_{41} + x_{51} + x_{62} = 5 + 2 + 4 + 6 + 5 + 4 = 26$$

$$\text{Communication Cost } (c) = c_{16} + c_{26} + c_{36} + c_{46} + c_{56} = 12 + 0 + 0 + 0 + 0 = 12$$

$$\text{Total Cost} = 26 + 12 = 38$$

Both theoretical calculations were incorporated as automated verification checks in the C test suite, confirming exact fidelity.

3 Parallelization Strategy using OpenMP

The dominant computational bottleneck in both metaheuristics is the evaluation of candidate allocations, which requires $O(M^2)$ pairwise communication penalty checks. When repeated for hundreds of individuals over hundreds of iterations, this evaluation dominates runtime.

- **Genetic Algorithm Parallelization:**

- Candidate chromosomes are mapped across threads using a persistent `#pragma omp parallel` region.
- In each generation, tournament selection, crossover, and mutation are computed in parallel across the population with `#pragma omp for schedule(static)`.
- Population fitness evaluations are computed concurrently with zero thread fork/join overhead.
- A thread-safe Linear Congruential Generator (LCG) with per-thread seeds prevents global lock contention.

- **Particle Swarm Optimization Parallelization:**

- Each particle maintains a position vector (processor mapping) and velocity vector.

- Velocity and position updates, boundary clamping, and $O(M^2)$ fitness evaluations are executed concurrently using `#pragma omp for schedule(static)`.
- Global best (*gbest*) updates across the swarm are coordinated via thread-safe master synchronizations.

4 C Implementation Code

The following is the self-contained C implementation (`task_alloc.c`):

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <limits.h>
4  #include <math.h>
5  #include <omp.h>
6  #include <string.h>
7
8  #define INF 999999
9  #define GA_POP_SIZE 120
10 #define GA_GENERATIONS 200
11 #define GA_MUTATION_RATE 0.05
12 #define GA_TOURNAMENT_SIZE 3
13
14 #define PSO_SWARM_SIZE 120
15 #define PSO_ITERATIONS 200
16 #define PSO_W 0.7
17 #define PSO_C1 1.5
18 #define PSO_C2 1.5
19 #define V_MAX 3.0
20
21 typedef struct {
22     int M, P;
23     int *exec_cost; // M x P matrix
24     int *comm_cost; // M x M matrix
25 } Problem;
26
27 typedef struct {
28     long long best_cost;
29     int *best_assignment;
30     double elapsed_time;
31 } Result;
32
33 // Cost calculation function
34 long long calculate_cost(const int assignment[], const Problem *prob) {
35     long long exec = 0, comm = 0;
36     int M = prob->M, P = prob->P;
37
38     for (int i = 0; i < M; i++)
39         exec += prob->exec_cost[i * P + assignment[i]];
40
41     for (int i = 0; i < M; i++) {
42         int node_i = assignment[i];
43         for (int j = i + 1; j < M; j++) {
44             if (node_i != assignment[j]) {
45                 int c = prob->comm_cost[i * M + j];
46                 if (c > 0) comm += c;
47             }
48         }
49     }
50     return exec + comm;
51 }
52
53 // Fast thread-safe pseudo-random number generator

```

```

54 static inline unsigned int lcg_rand(unsigned int *seed) {
55     *seed = *seed * 1103515245 + 12345;
56     return (*seed / 65536) % 32768;
57 }
58
59 static inline double lcg_rand_double(unsigned int *seed) {
60     return (double)lcg_rand(seed) / 32767.0;
61 }
62
63 // Parallel Genetic Algorithm with OpenMP
64 Result run_ga_parallel(const Problem *prob) {
65     int M = prob->M, P = prob->P;
66     int *population = (int*)malloc(GA_POP_SIZE * M * sizeof(int));
67     int *new_population = (int*)malloc(GA_POP_SIZE * M * sizeof(int));
68     long long *fitness = (long long*)malloc(GA_POP_SIZE * sizeof(long long));
69     int *global_best = (int*)malloc(M * sizeof(int));
70     long long global_best_cost = LLONG_MAX;
71
72     double t_start = omp_get_wtime();
73
74     #pragma omp parallel
75     {
76         int tid = omp_get_thread_num();
77         unsigned int seed = 42 + tid * 10007;
78
79         #pragma omp for schedule(static)
80         for (int i = 0; i < GA_POP_SIZE; i++)
81             for (int j = 0; j < M; j++)
82                 population[i * M + j] = lcg_rand(&seed) % P;
83
84         #pragma omp for schedule(static)
85         for (int i = 0; i < GA_POP_SIZE; i++)
86             fitness[i] = calculate_cost(&population[i * M], prob);
87
88         #pragma omp single
89         {
90             int best_idx = 0;
91             for (int i = 1; i < GA_POP_SIZE; i++)
92                 if (fitness[i] < fitness[best_idx]) best_idx = i;
93             global_best_cost = fitness[best_idx];
94             memcpy(global_best, &population[best_idx * M], M * sizeof(int));
95         }
96
97         for (int gen = 0; gen < GA_GENERATIONS; gen++) {
98             #pragma omp single
99             memcpy(&new_population[0], global_best, M * sizeof(int));
100
101             #pragma omp for schedule(static)
102             for (int i = 1; i < GA_POP_SIZE; i++) {
103                 int p1 = lcg_rand(&seed) % GA_POP_SIZE;
104                 for (int t = 1; t < GA_TOURNAMENT_SIZE; t++) {
105                     int cand = lcg_rand(&seed) % GA_POP_SIZE;
106                     if (fitness[cand] < fitness[p1]) p1 = cand;
107                 }
108                 int p2 = lcg_rand(&seed) % GA_POP_SIZE;
109                 for (int t = 1; t < GA_TOURNAMENT_SIZE; t++) {
110                     int cand = lcg_rand(&seed) % GA_POP_SIZE;
111                     if (fitness[cand] < fitness[p2]) p2 = cand;
112                 }
113
114                 int point = lcg_rand(&seed) % M;
115                 for (int j = 0; j < M; j++)
116                     new_population[i * M + j] = (j < point) ? population[p1 * M + j] : population[p2
117 * M + j];
118
119                 for (int j = 0; j < M; j++)

```

```

119         if (lcg_rand_double(&seed) < GA_MUTATION_RATE)
120             new_population[i * M + j] = lcg_rand(&seed) % P;
121     }
122
123     #pragma omp for schedule(static)
124     for (int i = 0; i < GA_POP_SIZE; i++)
125         memcpy(&population[i * M], &new_population[i * M], M * sizeof(int));
126
127     #pragma omp for schedule(static)
128     for (int i = 0; i < GA_POP_SIZE; i++)
129         fitness[i] = calculate_cost(&population[i * M], prob);
130
131     #pragma omp single
132     {
133         for (int i = 0; i < GA_POP_SIZE; i++) {
134             if (fitness[i] < global_best_cost) {
135                 global_best_cost = fitness[i];
136                 memcpy(global_best, &population[i * M], M * sizeof(int));
137             }
138         }
139     }
140 }
141 }
142
143 double t_end = omp_get_wtime();
144 free(population); free(new_population); free(fitness);
145
146 Result res;
147 res.best_cost = global_best_cost;
148 res.best_assignment = global_best;
149 res.elapsed_time = t_end - t_start;
150 return res;
151 }
152
153 // Parallel Particle Swarm Optimization with OpenMP
154 Result run_pso_parallel(const Problem *prob) {
155     int M = prob->M, P = prob->P;
156     int *pos = (int*)malloc(PSO_SWARM_SIZE * M * sizeof(int));
157     double *vel = (double*)malloc(PSO_SWARM_SIZE * M * sizeof(double));
158     int *pbest_pos = (int*)malloc(PSO_SWARM_SIZE * M * sizeof(int));
159     long long *pbest_cost = (long long*)malloc(PSO_SWARM_SIZE * sizeof(long long));
160     int *gbest_pos = (int*)malloc(M * sizeof(int));
161     long long gbest_cost = LLONG_MAX;
162
163     double t_start = omp_get_wtime();
164
165     #pragma omp parallel
166     {
167         int tid = omp_get_thread_num();
168         unsigned int seed = 777 + tid * 54321;
169
170         #pragma omp for schedule(static)
171         for (int i = 0; i < PSO_SWARM_SIZE; i++) {
172             for (int j = 0; j < M; j++) {
173                 pos[i * M + j] = lcg_rand(&seed) % P;
174                 vel[i * M + j] = (lcg_rand_double(&seed) * 2.0 - 1.0) * V_MAX;
175                 pbest_pos[i * M + j] = pos[i * M + j];
176             }
177             pbest_cost[i] = calculate_cost(&pos[i * M], prob);
178         }
179
180         #pragma omp single
181         {
182             for (int i = 0; i < PSO_SWARM_SIZE; i++) {
183                 if (pbest_cost[i] < gbest_cost) {
184                     gbest_cost = pbest_cost[i];

```

```

185         memcpy(gbest_pos, &pos[i * M], M * sizeof(int));
186     }
187 }
188
189
190 for (int iter = 0; iter < PSO_ITERATIONS; iter++) {
191     #pragma omp for schedule(static)
192     for (int i = 0; i < PSO_SWARM_SIZE; i++) {
193         for (int j = 0; j < M; j++) {
194             double r1 = lcg_rand_double(&seed);
195             double r2 = lcg_rand_double(&seed);
196
197             vel[i * M + j] = PSO_W * vel[i * M + j]
198                 + PSO_C1 * r1 * (pbest_pos[i * M + j] - pos[i * M + j])
199                 + PSO_C2 * r2 * (gbest_pos[j] - pos[i * M + j]);
200
201             if (vel[i * M + j] > V_MAX) vel[i * M + j] = V_MAX;
202             if (vel[i * M + j] < -V_MAX) vel[i * M + j] = -V_MAX;
203
204             int new_p = (int)round(pos[i * M + j] + vel[i * M + j]);
205             if (new_p < 0) new_p = 0;
206             if (new_p >= P) new_p = P - 1;
207             pos[i * M + j] = new_p;
208         }
209
210         long long current_cost = calculate_cost(&pos[i * M], prob);
211         if (current_cost < pbest_cost[i]) {
212             pbest_cost[i] = current_cost;
213             memcpy(&pbest_pos[i * M], &pos[i * M], M * sizeof(int));
214         }
215     }
216
217     #pragma omp single
218     {
219         for (int i = 0; i < PSO_SWARM_SIZE; i++) {
220             if (pbest_cost[i] < gbest_cost) {
221                 gbest_cost = pbest_cost[i];
222                 memcpy(gbest_pos, &pbest_pos[i * M], M * sizeof(int));
223             }
224         }
225     }
226 }
227
228
229 double t_end = omp_get_wtime();
230 free(pos); free(vel); free(pbest_pos); free(pbest_cost);
231
232 Result res;
233 res.best_cost = gbest_cost;
234 res.best_assignment = gbest_pos;
235 res.elapsed_time = t_end - t_start;
236 return res;
237 }

```

5 Experimental Results and Performance Comparison

Testing was conducted on a large problem instance with $M = 250$ tasks and $P = 8$ processors ($P \ll M$), execution costs between 10 and 100, and a 15% communication matrix density. The baseline random allocation total cost was 127,049.

Table 1: Comparative Benchmarking Results ($M = 250$, $P = 8$, 4 OpenMP Threads)

Algorithm	Time (sec)	Speedup	Best Cost	Cost Reduction
Sequential GA	1.7369 s	1.00× (base)	103,939	18.19%
Parallel GA (OpenMP)	0.6340 s	2.74×	105,818	16.71%
Sequential PSO	1.4537 s	1.00× (base)	101,446	20.15%
Parallel PSO (OpenMP)	0.6320 s	2.30×	101,587	20.04%

6 Execution Output

```
=====
VERIFICATION: Sample 6-Task, 2-Node Distributed Topology
=====
Serial Assignment Cost   : Calculated = 58 | Expected = 58 -> PASSED
Optimal Assignment Cost  : Calculated = 38 | Expected = 38 -> PASSED
=====
TASK ALLOCATION PROBLEM (TAP) BENCHMARK
Dimensions: M = 250 Tasks, P = 8 Processors (P << M)
OpenMP Threads Available: 4
=====
Generating synthetic problem instance (Exec Cost: 10-100, Comm Density: 15%)...
Random Baseline Allocation Total Cost: 127049

--- Running Genetic Algorithm (Pop: 120, Gens: 200) ---
Running Sequential GA...
  Sequential GA Best Cost : 103939 | Time: 1.7369 s
Running Parallel OpenMP GA (4 threads)...
  Parallel GA Best Cost   : 105818 | Time: 0.6340 s | Speedup: 2.74x

--- Running Particle Swarm Optimization (Swarm: 120, Iters: 200) ---
Running Sequential PSO...
  Sequential PSO Best Cost: 101446 | Time: 1.4537 s
Running Parallel OpenMP PSO (4 threads)...
  Parallel PSO Best Cost  : 101587 | Time: 0.6320 s | Speedup: 2.30x

=====
Algorithm          | Time (sec) | Speedup | Best Cost | Cost Reduction
=====
Sequential GA      | 1.7369 s | 1.00x (base) | 103939 | 18.19%
Parallel GA (OpenMP) | 0.6340 s | 2.74x | 105818 | 16.71%
Sequential PSO     | 1.4537 s | 1.00x (base) | 101446 | 20.15%
Parallel PSO (OpenMP) | 0.6320 s | 2.30x | 101587 | 20.04%
=====

Sample Task-to-Processor Mapping (First 20 Tasks from Parallel PSO, Total Cost = 101587):
Task 1 -> Processor 4
Task 2 -> Processor 4
Task 3 -> Processor 4
Task 4 -> Processor 4
Task 5 -> Processor 4
Task 6 -> Processor 8
Task 7 -> Processor 3
Task 8 -> Processor 1
Task 9 -> Processor 5
Task 10 -> Processor 4
... [230 more tasks mapped]
```

7 Conclusion

The parallel implementations of Genetic Algorithm and Particle Swarm Optimization on OpenMP effectively resolved the computational overhead of $O(M^2)$ cross-node communication evaluations. On 4 cores, Parallel GA achieved a **2.74× speedup** and Parallel PSO achieved a **2.30× speedup**, while both converged to allocations achieving approximately 17–20% lower cost than random baseline mappings.